RESUMEN P2 – ISW

UNIDAD 2: GESTIÓN LEAN AGILE DE PRODUCTOS DE SOFTWARE

1. FRAMEWORK PARA ESCALAR SCRUM / NEXUS

La necesidad de **escalar Scrum** surge cuando varios equipos deben trabajar sobre un mismo producto y necesitan coordinarse. Al aumentar la cantidad de equipos, aparecen dependencias entre sus tareas, dificultades de comunicación y problemas para integrar lo que desarrolla cada uno. No alcanza con que cada equipo aplique Scrum por separado: es necesario alinear sus esfuerzos hacia un objetivo común y lograr que sus resultados formen **un incremento integrado, útil y de calidad**. Frameworks como **Nexus** permiten gestionar esa coordinación y reducir las dependencias entre equipos.

Framework Nexus

Nexus es un framework para escalar Scrum que coordina aproximadamente **tres a nueve Scrum Teams** sobre un mismo producto. Comparten un **Product Owner**, un **Product Backlog** y producen un **incremento integrado** por Sprint. Su propósito es reducir las dependencias y los problemas de coordinación e integración, manteniendo la autogestión y el empirismo de Scrum. La complejidad de vincularlos radica en:

1. Requerimientos: el alcance de estos puede superponerse. La forma en que se implementan también puede afectar a los demás.
2. Conocimiento del dominio: el conocimiento del sistema de negocio debería mapearse a los Equipos Scrum para minimizar las interrupciones entre los mismos durante el Sprint.

Roles y responsabilidades

Nexus mantiene las responsabilidades de Scrum y agrega el **Nexus Integration Team**, encargado de asegurar que el trabajo de los distintos equipos forme un incremento integrado y terminado al menos una vez por Sprint.

El **Product Owner** es único para todo el Nexus. Es responsable de maximizar el valor del producto y de gestionar eficazmente el **Product Backlog compartido**, ordenando sus elementos y orientando el trabajo hacia un objetivo de producto común.

El **Scrum Master que participa en el Nexus Integration Team** es responsable de ayudar a comprender y aplicar el framework Nexus. Puede desempeñarse también como Scrum Master de uno o más equipos Scrum.

Los **miembros del Nexus Integration Team** aportan los conocimientos necesarios para identificar dependencias, abordar impedimentos y ayudar a adoptar prácticas y herramientas de integración. Su composición puede cambiar según las necesidades del producto. Los **Scrum Teams** conservan su autogestión y realizan el desarrollo y la integración del trabajo.
El Nexus Integration Team proporciona orientación y un foco de responsabilidad sobre la integración; no funciona como un equipo separado al que se entrega todo el trabajo para integrarlo al final.

Eventos y refinamiento

El **Sprint** constituye el período común de trabajo del Nexus. Durante él, los equipos colaboran para alcanzar un objetivo compartido y producir un incremento integrado.
El **Refinamiento entre equipos** es una actividad continua destinada a aclarar y descomponer los elementos del Product Backlog. Permite prever qué equipo podría desarrollar cada elemento e identificar dependencias para eliminarlas o reducirlas antes de comenzar el trabajo. Su frecuencia y participación se adaptan a las necesidades.
La **Nexus Sprint Planning** coordina el trabajo de todos los equipos para el Sprint. Participan el Product Owner y representantes adecuados de los Scrum Teams. Se define el **Nexus Sprint Goal**, los objetivos de Sprint de cada equipo alineados con él, el Nexus Sprint Backlog y los Sprint Backlogs individuales.
La **Nexus Daily Scrum** permite inspeccionar el progreso hacia el objetivo común y detectar problemas de integración o nuevas dependencias. Participan representantes adecuados de los equipos. Las Daily Scrums individuales complementan este evento, adaptando sus planes diarios para atender los problemas identificados. La comunicación entre equipos continúa durante el resto del día.
La **Nexus Sprint Review** permite inspeccionar el incremento integrado junto con los interesados, obtener retroalimentación y determinar futuras adaptaciones del producto. **Reemplaza las Sprint Reviews individuales**, porque el resultado que se evalúa es el producto integrado.
La **Nexus Sprint Retrospective** busca mejorar la calidad y la eficacia del conjunto. Se analizan las interacciones entre equipos, los procesos, las herramientas y la Definition of Done. Las retrospectivas individuales se mantienen y complementan esta reflexión, aportando problemas y propuestas que afectan a todo el Nexus.

Artefactos y compromisos

El **Product Backlog** es único y contiene el trabajo necesario para mejorar el producto. Debe permitir comprender los elementos e identificar sus dependencias. Su compromiso es el **Product Goal**, que describe el estado futuro del producto y orienta el trabajo de todos los equipos.
El **Nexus Sprint Backlog** reúne el Nexus Sprint Goal y los elementos seleccionados en los Sprint Backlogs de los equipos. Hace visibles las dependencias y el flujo de trabajo durante el Sprint, y se actualiza a medida que se aprende. Su compromiso es el **Nexus Sprint Goal**, que da coherencia y foco al esfuerzo conjunto. Cada equipo conserva además su propio Sprint Backlog.
El **Integrated Increment** es el resultado de todo el trabajo completado e integrado del Nexus. Debe ser útil, utilizable y cumplir las condiciones de calidad acordadas. Puede entregarse antes de la Sprint Review. Su compromiso es la **Definition of Done**, común para todos los equipos. Un equipo puede establecer criterios más exigentes, pero nunca menos rigurosos que los acordados para el incremento integrado.

2. FILOSOFÍA LEAN

Relación entre Lean y la filosofía Agile: ambas priorizan la entrega de valor al cliente, el aprendizaje mediante retroalimentación, la colaboración y la capacidad de adaptarse a los cambios. Agile aporta valores y principios para desarrollar software de manera iterativa e incremental; Lean se concentra en optimizar el flujo completo, eliminar desperdicios y mejorar continuamente el sistema de trabajo. No son enfoques opuestos: un equipo Scrum puede aplicar principios Lean y prácticas Kanban para limitar el trabajo en curso, reducir esperas y acelerar la entrega sin descuidar la calidad.
Lean es una filosofía de gestión, originada en el sistema de producción de Toyota, que busca **maximizar el valor entregado al cliente y reducir el desperdicio**. En desarrollo de software, implica concentrar el trabajo en resolver necesidades reales y mejorar el proceso completo, desde que surge una necesidad hasta que el usuario recibe una solución.
El **valor** se determina desde la perspectiva del cliente: una actividad aporta valor cuando contribuye a obtener un producto que satisface sus necesidades. El **desperdicio** es el consumo de tiempo, esfuerzo o recursos que no contribuye a ese resultado.

Lean también busca mejorar el **flujo de trabajo**: reducir esperas, acumulaciones y retrabajo para entregar valor con mayor rapidez. Por eso propone observar el sistema completo; acelerar la programación sirve poco si el software permanece semanas esperando pruebas o despliegue. Lean Software Development - An ...

Los siete principios de Lean aplicados al software

1. **Eliminar desperdicios.** Consiste en identificar y reducir actividades que consumen recursos sin aportar valor. Por ejemplo, desarrollar funcionalidades innecesarias, mantener trabajo incompleto o duplicar documentación. Primero hay que comprender para qué sirve una actividad: una prueba o un documento necesario no debe eliminarse solo porque no produce código.
2. **Amplificar el aprendizaje.** El desarrollo de software requiere descubrir necesidades y encontrar soluciones. Lean propone aprender mediante ciclos cortos, feedback, pruebas y experimentos, compartiendo el conocimiento dentro del equipo. Mostrar una versión pequeña al usuario permite corregir interpretaciones antes de construir una solución completa.
3. **Embeber la integridad conceptual.** Significa construir la calidad y la coherencia del producto desde el comienzo. Sus componentes deben funcionar como un conjunto y ofrecer una experiencia consistente con lo que necesita el usuario. Esto exige cuidar el diseño, integrar frecuentemente, probar y refactorizar; la calidad se trabaja durante el desarrollo.
4. **Diferir compromisos hasta el último momento responsable.** Consiste en postergar decisiones difíciles de revertir mientras todavía resulte conveniente obtener más información. Se mantienen alternativas abiertas y se decide antes de que la demora haga perder opciones o genere perjuicios. No significa dejar todo para último momento.
5. **Dar poder al equipo.** Las personas que realizan el trabajo deben participar en las decisiones sobre cómo llevarlo adelante. Para ello necesitan conocimientos, información, objetivos claros y autonomía. Los líderes brindan orientación y eliminan obstáculos, aprovechando la experiencia de los integrantes.
6. **Ver el todo.** Propone optimizar el proceso completo y el resultado para el cliente, evitando mejoras locales que perjudiquen al sistema. Por ejemplo, aumentar la cantidad de funcionalidades programadas puede ser contraproducente si supera la capacidad de pruebas y acumula trabajo sin terminar.
7. **Entregar lo antes posible.** Trabajar en lotes pequeños y reducir demoras permite obtener valor y retroalimentación tempranamente. Las entregas frecuentes también facilitan detectar problemas y adaptarse a cambios. La rapidez se consigue mejorando el flujo y reduciendo desperdicios, manteniendo la calidad.

Tipos de desperdicio: La clasificación utilizada en la filmina distingue:

- **Tipo 1 — Desperdicio necesario:** no agrega valor directamente para el cliente, pero resulta necesario bajo las condiciones actuales. Se busca **minimizarlo** y encontrar formas más eficientes de realizarlo. Por ejemplo, una carga administrativa obligatoria que podría simplificarse o automatizarse.
- **Tipo 2 — Puro desperdicio:** no agrega valor ni es necesario. Se busca **evitarlo o eliminarlo**. Por ejemplo, registrar la misma información en dos lugares cuando uno de ellos no tiene ninguna utilidad.

La distinción exige analizar el contexto. Los Poppendieck señalan que la documentación puede ser desperdicio cuando nadie la necesita, pero también puede aportar valor cuando ayuda a comprender, construir, probar o cumplir requisitos relevantes. **No toda actividad de soporte es desperdicio.**

Los siete desperdicios en el desarrollo de software

1. **Trabajo parcialmente terminado — Inventario.** Es trabajo iniciado que todavía no puede generar el valor esperado: requisitos sin implementar, código sin integrar, funcionalidades sin probar o cambios pendientes de entrega. Acumula inversión sin obtener resultados y puede volverse obsoleto. Se reduce trabajando en lotes pequeños y terminando antes de comenzar más.
2. **Características extra — Sobreproducción.** Son funcionalidades desarrolladas sin una necesidad real que las justifique. Además del esfuerzo inicial, requieren pruebas, mantenimiento y aumentan la complejidad. Por ejemplo, agregar opciones “por si algún día se necesitan”, sin evidencia de su utilidad.
3. **Procesos extra — Sobreprocesamiento.** Son pasos extra, documentos o controles que no aportan valor suficiente. Por ejemplo, repetir aprobaciones o elaborar informes que nadie utiliza. Se debe revisar su propósito y simplificarlos, conservando los controles necesarios.
4. **Transporte - Cambio de tareas / Transferencia de conocimiento.** Ocurre cuando una persona alterna continuamente entre actividades o proyectos. Cada cambio exige recuperar información y concentración, lo que aumenta el tiempo total y favorece errores. Por ejemplo, interrumpir repetidamente una funcionalidad para atender tareas diferentes.
5. **Esperas o demoras.** Son períodos en los que el trabajo no avanza porque falta información, una aprobación, un entorno o el resultado de otra actividad. Retrasan la entrega de valor al cliente. Por ejemplo, una funcionalidad terminada que espera varios días para ser probada.
6. **Movimiento y búsqueda de información.** Comprende el esfuerzo innecesario para encontrar respuestas, acceder a recursos o trasladar información entre personas y áreas. Los traspasos también pueden perder conocimiento que no quedó documentado. Por ejemplo, buscar repetidamente una decisión o volver a explicar un requisito a cada área que recibe el trabajo.
7. **Defectos.** Son problemas que obligan a investigar, corregir y repetir actividades. Su impacto puede crecer cuanto más tiempo permanecen sin detectarse. Una regla de negocio mal interpretada puede afectar código, pruebas y documentación. La prevención, las pruebas tempranas y la integración frecuente reducen este desperdicio.
8. Talento no utilizado. Se desperdician los conocimientos, la creatividad y la experiencia del equipo cuando no se escuchan propuestas, no se delegan decisiones o se asignan tareas sin aprovechar las capacidades de las personas. Por ejemplo, mantener a buenos programadores en tareas rutinarias sin oportunidades de aprendizaje, participación o desarrollo profesional puede desaprovechar su potencial y disminuir la motivación. La mejora exige escuchar al equipo, brindar autonomía y favorecer su crecimiento.

Las denominaciones y agrupaciones cambian entre fuentes. En *Implementing Lean Software Development*, los autores reorganizan la lista e incluyen **reaprendizaje** y **traspasos** explícitamente. La finalidad de clasificar es reconocer dónde se pierde valor y mejorar el trabajo. La filmina agrega además un octavo desperdicio: **talento no utilizado**, cuando no se aprovechan los conocimientos, capacidades e ideas de las personas.

3. MÉTODO KANBAN EN EL DESARROLLO DE SOFTWARE

Método KANBAN: Kanban constituye una metodología orientada a definir, administrar y perfeccionar la prestación de servicios en tareas de conocimiento, tales como el desarrollo de software. Su fin primordial radica en optimizar el flujo operativo tornando el avance fluido y predecible, al tiempo que busca armonizar las demandas recibidas con la capacidad operativa disponible.

Con este propósito, la disciplina propone visibilizar la totalidad del flujo, restringir la concurrencia de tareas iniciadas y emplear la métrica del proceso para impulsar la mejora continua. Se parte de las prácticas vigentes, incorporando transformaciones graduales fundadas en los problemas y oportunidades identificados.

Aun cuando una herramienta gráfica expone el flujo, **el método trasciende el simple tablero**: contempla la restricción de trabajo en proceso, explicitación de reglas, gestión activa del avance y evolución constante. David Anderson contrapone un simple panel visual frente a un auténtico

sistema kanban, caracterizando a este último por incorporar límites estrictos y señales claras de capacidad libre para incorporar nuevos compromisos.

Origen: Kanban y Just in Time: El término oriental *kanban* alude a una señal visual, tarjeta o letrero indicador. Sus raíces industriales remiten directamente al esquema de producción implementado por Toyota, donde dichos avisos regulaban el reabastecimiento limitando las existencias en curso.

Esta visión se enlaza con el paradigma **Just in Time**, tendiente a elaborar únicamente lo indispensable en el momento oportuno, anulando acumulaciones inconclusas. En el terreno informático, la dinámica se ajusta posibilitando halar nuevo trabajo tan pronto como se libere espacio operativo. Las tarjetas simbolizan entregables concretos, mientras los umbrales prefijados gobiernan su incorporación.

Valores de Kanban

Los principios axiológicos del esquema orientan la dinámica de colaboración e implementación:

- **Transparencia:** exhibir abiertamente datos del trabajo, reglas y restricciones para respaldar resoluciones acertadas.
- **Equilibrio:** equiparar demandas externas, capacidad real y requerimientos diversos para prevenir el colapso del equipo.
- **Colaboración:** aunar esfuerzos colectivos enfocado en optimizar el servicio.
- **Foco en el cliente:** alinear las decisiones del sistema con las necesidades y el valor real percibido por el destinatario.
- **Flujo:** comprender cabalmente el recorrido de las tareas hasta materializarse en entregas de valor.
    - Diferencia principal con Scrum: Scrum estructura el trabajo en Sprints de duración fija, con responsabilidades, eventos y artefactos definidos para obtener un incremento de producto. Kanban no exige Sprints ni establece esos roles o eventos; gestiona un flujo continuo de trabajo mediante un sistema visual, límites WIP, políticas explícitas y métricas como Lead Time y Cycle Time. Ambos pueden combinarse para mejorar el flujo y la previsibilidad.
- **Liderazgo:** incentivar iniciativas de mejora continua sin distinción de jerarquías.
- **Entendimiento:** analizar la realidad actual en profundidad antes de procurar transformaciones.
- **Acuerdo:** forjar consensos genuinos hacia metas y objetivos unificados.
- **Respeto:** valorar los aportes, la trayectoria y el rol de cada participante.

Principios de Kanban

Principios de gestión del cambio

Comenzar con lo que se hace actualmente. Supone asimilar el proceso en marcha respetando la estructura de roles y prácticas efectivas, minimizando de este modo los niveles de fricción institucional.

Acordar la mejora mediante el cambio evolutivo. Implica instaurar ajustes graduales examinando los impactos reales, de forma tal que la dinámica progrese basada en evidencia concreta.

Fomentar actos de liderazgo en todos los niveles. Todo integrante posee la facultad de detectar ineficiencias y articular sugerencias, posibilitando que la iniciativa surja tanto de la base operativa como de la línea directiva.

Principios de entrega de servicios

Comprender y atender las necesidades y expectativas del cliente. La eficacia global de la prestación se mide en función de su aptitud para proporcionar soluciones satisfactorias.

Gestionar el trabajo y permitir la autoorganización de las personas. La prioridad estriba en monitorear el curso de las tareas e identificar trabas, facultando al equipo a articularse dinámicamente según tales necesidades.

Revisar y evolucionar las políticas del servicio. Requiere evaluar periódicamente las normativas operativas para optimizar el valor global del negocio, impidiendo sesgos locales desajustados.

Prácticas generales de Kanban

1. Visualizar el trabajo

Consiste en esquematizar el recorrido de las tareas a través de un panel donde las columnas reflejan las fases operativas y las tarjetas representan los elementos de trabajo requeridos.

Tal esquema permite explicitar actividades pendientes, esfuerzos en curso, demoras o bloqueos, consolidando un entendimiento alineado que potencia la coordinación.

2. Limitar el trabajo en progreso — WIP

El **WIP (*Work in Progress*)** representa la carga operativa admitida en el circuito pero aún no concluida, contemplando ítems en ejecución activa o en estado de espera.

Imponer cotas implica normar la cantidad máxima de ítems tolerados por columna. Al alcanzar dicho techo, la directiva exige finalizar tareas pendientes antes de aceptar nuevos compromisos.

Esta práctica atenúa la dispersión de esfuerzos e indudablemente expone los cuellos de botella: si una fase se satura, resulta imprescindible diagnosticar sus causas subyacentes.

3. Gestionar el flujo

Reside en supervisar la fluidez del trabajo procurando acortar lapsos muertos, destrabar detenciones y dotar de predecibilidad a las fechas de entrega.

El eje medular apunta a **completar unidades funcionales**; de ahí que saturar de ocupación a la nómina no asegure rendimiento, pudiendo generar colas de espera indeseadas.

4. Hacer explícitas las políticas

Las reglas definen las condiciones para la selección, limites de WIP e idoneidad de pasaje entre etapas o criterios de cierre definitivo.

Deben caracterizarse por su concisión, visibilidad y capacidad de revisión continua, favoreciendo resoluciones autónomas sustentadas en acuerdos preestablecidos.

5. Implementar circuitos de retroalimentación

Exige monitorear de manera regular el desempeño del servicio, confrontando resultados reales frente a las expectativas del cliente para articular ajustes.

Estas instancias se efectivizan mediante encuentros focalizados en el estado diario, reabastecimiento, coordinación de entregas o balance general, estructurando las denominadas **cadencias de Kanban**.

6. Mejorar colaborativamente y evolucionar experimentalmente

El equipo examina falencias y plantea variaciones bajo la forma de experimentos guiados por hipótesis, evaluando objetivamente el resultado final. Un ejemplo válido consiste en restringir el WIP para medir la reducción de demoras; de resultar favorable, la pauta se consolida, caso contrario se recalibra basándose en el aprendizaje grupal.

Administración de colas y sistema pull

Las **colas** albergan ítems a la espera de procesamiento. Su ordenamiento implica controlar el volumen, criterios de prioridad y tiempo de permanencia; la acumulación desmedida dilata los plazos e incrementa el costo de oportunidad.

La metodología recurre a un **esquema de tracción (pull)**: cada fase absorbe tareas únicamente cuando cuenta con vacantes disponibles. En contraste, un modelo de empuje (*push*) recarga las estaciones en función de la demanda entrante, sin reparar en su nivel de saturación.

Por caso, si la instancia de testing alcanza su capacidad límite, el equipo debe colaborar para destrabar dicho bloque. Adicionar más funcionalidades inconclusas incrementa el volumen de inventario y demora la salida al mercado.

¿Cómo aplicar Kanban?

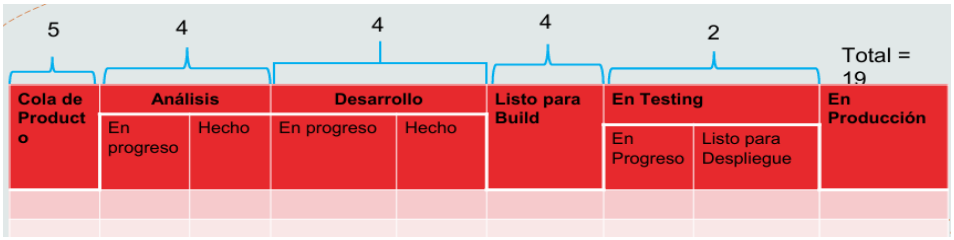
Modelar el proceso. Trazar las fases reales que atraviesan los requerimientos fijando los hitos inicial y final (ej. análisis, desarrollo, pruebas, despliegue), diferenciando labores activas e instancias de espera.



Definir las unidades y los tipos de trabajo. Precisar la naturaleza de las tarjetas identificando tipologías de demanda como historias de usuario, fallos, mantenimiento preventivo o cambios, balanceando la capacidad disponible.



Establecer límites de WIP. Pactar techos máximos de tareas simultáneas por etapa y ajustarlos progresivamente según las métricas operativas.



Definir políticas de calidad y funcionamiento. Explicitar pautas de ingreso, transición y finalización, normando la selección y los criterios de aceptación.

Gestionar cuellos de botella. Ubicar los puntos de estancamiento y accionar sobre sus causas primarias, evaluando el impacto de cada ajuste sobre la globalidad del proceso.

Definir clases de servicio y cadencias. Formular esquemas de atención diferenciados según urgencias y estructurar la frecuencia de revisión y reabastecimiento adaptándola al entorno.

Clases de servicio

Las clases de servicio permiten diferenciar el tratamiento asignado a las tareas según **el costo de diferir su concreción**, el cual cuantifica el impacto financiero o perjuicio por demoras.

Expreso o urgente. Aplica a tareas cuyo aplazamiento ocasiona daños críticos e inmediatos, como caídas del entorno productivo. Goza de máxima prioridad e interrumpe actividades en curso, por lo que su uso requiere estricta dosificación.

Fecha fija. Reagrupa requerimientos subordinados a un límite temporal estricto, vencido el cual las penalizaciones crecen drásticamente. Exige planificación anticipada.

Estándar. Abarca los requerimientos habituales cuyo diferimiento posterga beneficios sin causar emergencias, procesándose mediante reglas ordinarias.

Intangible. Comprende tareas de valor evidente pero con bajo impacto de demora inmediato, cuyo costo escala a largo plazo (e.g. refactorización de infraestructura). Conviene reservar capacidad para evitar su postergación crónica.

Las convenciones de color, prioridades específicas y concesiones sobre el WIP integran **políticas concertadas según el contexto de aplicación**.

Cadencias

Cadencias destacadas en la filmina de la cátedra (Lean y Kanban, p. 42): Team Kanban Meeting, diaria, para observar el estado y el flujo del trabajo, detectar bloqueos y decidir qué hacer a continuación; Team Retrospective, quincenal o mensual, para reflexionar sobre la forma de trabajar y mejorarla; y Team Replenishment Meeting, semanal o a demanda, para seleccionar los próximos elementos a incorporar al sistema. Más adelante se conserva el listado ampliado de siete cadencias que ya figuraba en este resumen.

Las cadencias constituyen reuniones periódicas destinadas a la alineación operativa y retroalimentación, cuya frecuencia se amolda al contexto podendo articularse sobre rutinas prefijadas.

La literatura estándar identifica siete instancias fundamentales:

- **Revisión de estrategia:** encauzar el rumbo de los servicios en función de la visión del negocio.
- **Revisión de operaciones:** sincronizar diferentes células de trabajo balanceando carga y capacidad.

- **Revisión de riesgos:** evaluar contingencias y destrabar cuellos de botella para garantizar entregas.
- **Revisión de prestación del servicio:** medir la performance obtenida e instituir mejoras.
- **Reunión de reposición:** acordar los nuevos requerimientos a incorporar en el flujo.
- **Reunión Kanban:** monitorear el avance diario, resolver bloqueos y alinear acciones inmediatas.
- **Planificación de entregas:** coordinar y estructurar el despliegue de soluciones al cliente.

Relación Kanban-Scrum

Scrum organiza el trabajo mediante Sprints, responsabilidades, eventos y artefactos orientados a producir incrementos utilizables. Kanban se concentra en gestionar y mejorar el flujo mediante visualización, límites de trabajo en progreso y políticas explícitas, sin exigir iteraciones de duración fija.

Ambos favorecen la transparencia, la retroalimentación y la mejora continua. Pueden combinarse: un equipo puede conservar los Sprints y eventos de Scrum e incorporar límites de WIP, gestión de bloqueos y métricas de flujo de Kanban.

Métricas de Kanban

Lead Time (Tiempo de entrega)

Comprende el intervalo temporal total transcurrido desde que la demanda es formulada por el usuario hasta la recepción efectiva del producto finalizado, abarcando las latencias intermedias.

Se diferencia asimismo el **tiempo de entrega del sistema**, formalizado desde la asunción explícita del compromiso operativo hasta su conclusión; de allí la necesidad de explicitar los hitos delimitadores del cálculo.

Cycle Time (Tiempo de ciclo)

Contabiliza la duración requerida desde la iniciación formal del desarrollo en el circuito hasta su culminación definitiva, ofreciendo un indicador de desempeño interno.

Incorpora las demoras y bloqueos suscitados; no equivale al trabajo neto volcado. Una tarea puede requerir dos jornadas de desarrollo activo pero registrar seis días de ciclo al quedar estancada en verificación.

Touch Time (Tiempo de trabajo activo)

Representa la fracción temporal exacta durante la cual se volcaron esfuerzos efectivos de elaboración, aislando el trabajo real frente a lapsos ociosos o retenciones.

Eficiencia del ciclo del proceso

Proporción porcentual que mide la dedicación efectiva sobre la totalidad del tiempo transcurrido:

Eficiencia = (Touch Time / Tiempo Total Transcurrido) × 100

Dichas variables deben homogeneizarse en la misma unidad temporal. Por caso, si un requerimiento absorbió dos días de esfuerzo activo en un período global de diez días, exhibirá una eficiencia del **20 %**, permitiendo focalizar la supresión de tiempos muertos.

WIP y tasa de entrega

El **WIP** cuantifica la masa de elementos abordados aún inconclusos. En contrapartida, la **tasa de entrega o throughput** explicita la cantidad de entregables finalizados por unidad de tiempo definida (e.g. ocho requerimientos semanales).

Los registros históricos de velocidad e intervalo facultan proyecciones operativas; resulta prioritario estudiar su dispersión, dado que una media simple puede enmascarar variaciones significativas según la tipología del encargo.

Ley de Little: en un sistema estable, WIP promedio = Throughput promedio × Cycle Time promedio. Por lo tanto, Cycle Time promedio = WIP promedio / Throughput promedio. Ejemplo ilustrativo: si hay en promedio 8 elementos en curso y se completan 4 elementos por semana, el tiempo de ciclo promedio será de 2 semanas (8 / 4). Es importante usar los mismos límites de entrada y salida al medir las tres magnitudes, emplear unidades compatibles y recordar que la relación se aplica a promedios de un sistema suficientemente estable.

4. MÉTRICAS EN LOS TRES ENFOQUES (TRADICIONAL / LEAN / AGILE)

Métricas en el enfoque tradicional

Destinadas a **supervisar el apego a la planificación e inspeccionar la calidad**, cotejando las previsiones frente al avance real. Se agrupan en indicadores de proceso, proyecto y producto, destacándose:

- **Esfuerzo:** carga laboral insumida, expresada habitualmente en horas-persona.
- **Tiempo:** extensión cronológica y verificación frente a los hitos proyectados.
- **Costos:** egresos financieros consolidados respecto al presupuesto asignado.
- **Defectos:** volumen, severidad y localización de las anomalías detectadas.
- **Tamaño:** magnitud del sistema calculada en líneas de código o puntos de función.

Métricas en el enfoque Lean

Se orientan principalmente a **agilizar el flujo operativo y mitigar improductividades**, especialmente los tiempos muertos:

- **Lead Time:** plazo transcurrido desde la demanda inicial hasta la entrega efectiva.
- **Cycle Time:** lapso computado desde el inicio del trabajo hasta su finalización, integrando las esperas del proceso.
- **Touch Time:** duración en que la tarea recibió atención y desarrollo activo.
- **Eficiencia del proceso:** proporción del tiempo dedicado a la labor efectiva: **Touch Time / Tiempo Transcurrido × 100**.

Métricas en el enfoque Agile

Procuran **evaluar el avance mediante software operativo y respaldar la adaptación**. Sobresalen:

- **Velocidad:** volumen de trabajo completado por Sprint, habitualmente ponderado en puntos de historia.
- **Capacidad:** disponibilidad real del equipo para encarar la siguiente iteración, considerando la nómina, licencias y compromisos paralelos.
- **Running Tested Features (RTF):** cantidad de funcionalidades correctamente integradas que superaron las instancias de validación.

Capacidad y velocidad distan de ser equivalentes: la primera refleja el potencial de trabajo disponible; la segunda registra las entregas efectivamente concluidas. Aunque los esquemas pueden compartir métricas, difieren en su finalidad y en el foco del análisis.

UNIDAD 3: GESTIÓN DE SOFTWARE COMO PRODUCTO

1. PRÁCTICAS CONTINUAS

Las metodologías continuas permiten **incorporar, validar y liberar modificaciones pequeñas de manera recurrente** a lo largo del ciclo del producto. Su propósito radica en identificar inconsistencias de forma temprana, mitigar fricciones en la integración y acelerar la retroalimentación. Estas disciplinas se articulan mediante repositorios centralizados, automatización de procesos y ejecuciones de prueba durante el desarrollo.

Integración continua — Continuous Integration (CI)

Implica consolidar con frecuencia las contribuciones de los desarrolladores sobre un tronco común. Cada incorporación dispara la construcción automatizada de la solución y la corrida de pruebas para advertir divergencias o fallas a tiempo. Si se produce una falla, la prioridad máxima consiste en resolverla para asegurar una línea base estable. **Integrar no equivale a un despliegue productivo.**

Entrega continua — Continuous Delivery

Asegurar que el software permanezca **disponible para su puesta en producción en el momento en que el negocio lo requiera**. Extiende la integración continua incorporando la infraestructura de verificación necesaria para empaquetar una release. La decisión final de liberar el artefacto puede requerir una autorización deliberada.

Despliegue continuo — Continuous Deployment

Aquellas modificaciones que superan con éxito los filtros de validación se **publican automáticamente en el entorno productivo**, omitiendo aprobaciones manuales intermedias. El rasgo distintivo frente a la entrega continua estriba precisamente en la automatización completa del paso final.

Pipeline: secuencia automatizada que obtiene el código, construye el software, ejecuta controles y pruebas y genera un artefacto identificado. Puede incorporar preparación y despliegue según se utilice entrega o despliegue continuo. Sus etapas dependen del producto y sus riesgos.

Pruebas y automatización

Las instancias de testing acompañan la evolución del código para identificar fallas y corroborar que las capacidades operativas conserven su integridad. La automatización del ensamblaje, las verificaciones sistemáticas y la distribución minimizan la intervención manual no programada, acelerando el ciclo de retorno de información, complementándose con análisis exploratorios y revisiones humanas.

Estrategias de despliegue

Canary: habilita la versión renovada inicialmente a una fracción acotada del público o del flujo de tráfico. Se monitorea la respuesta del sistema y se escala progresivamente, conteniendo el rango de impacto ante posibles contingencias.

Blue-Green: mantiene dos entornos espejados: mientras uno atiende el tráfico activo, el otro recibe la actualización. Una vez validada la versión, se conmuta el flujo operativo hacia el ambiente actualizado. Esta infraestructura simplifica eventuales reversiones, en la medida en que la compatibilidad de datos lo permita.

UNIDAD 4: ASEGURAMIENTO DE CALIDAD DE PROCESO Y DE PRODUCTO

1. CONCEPTOS GENERALES SOBRE CALIDAD

¿Qué es la calidad del software?

La calidad del software es el grado en que un producto **satisface los requisitos y las necesidades explícitas e implícitas de sus interesados**, dentro de un contexto de uso. Comprende tanto lo que el sistema hace como la manera en que lo hace.

Las necesidades **explícitas** son las expresadas y documentadas, como registrar una reserva. Las **implícitas** son expectativas que pueden no haberse detallado, como conservar los datos, proteger la información o resultar comprensible para el usuario.

Por lo tanto, cumplir las funcionalidades solicitadas es necesario, pero también deben considerarse atributos como confiabilidad, seguridad, rendimiento, usabilidad y mantenibilidad. Un sistema puede calcular correctamente y aun resultar inadecuado si responde demasiado lento o es difícil de utilizar.

Perspectivas de la calidad

Pressman presenta cinco perspectivas que ayudan a comprender por qué diferentes interesados pueden valorar de distinta manera un mismo producto:

Perspectiva	Cómo entiende la calidad
Trascendental	Como una excelencia que se reconoce, aunque resulte difícil definirla.
Del usuario	Como la capacidad de satisfacer sus necesidades y objetivos.
De manufactura o conformidad	Como el cumplimiento de las especificaciones y estándares establecidos.
Del producto	Como las características y propiedades que posee el producto.
Basada en el valor	Como la relación entre los beneficios obtenidos y el costo que se está dispuesto a pagar.

Estas perspectivas se complementan. Para evaluar la calidad hay que identificar **para quién debe tener calidad el producto y qué atributos son relevantes en ese contexto**.

Calidad de proceso y calidad de producto

La **calidad de proceso** se refiere a la adecuación y eficacia de la forma de trabajar: actividades, responsabilidades, métodos, controles y herramientas utilizados para desarrollar y mantener software.

La **calidad de producto** se refiere a las características de los resultados obtenidos: requisitos, diseños, código, documentación y software funcionando.

Un proceso adecuado ayuda a prevenir defectos y producir resultados más consistentes, pero **su cumplimiento no garantiza automáticamente un producto de calidad**. También debe evaluarse el resultado y comprobarse que responde a las necesidades reales.

Gestión, aseguramiento, planificación y control de calidad

La **gestión de calidad** comprende las actividades destinadas a establecer y alcanzar los objetivos de calidad. Dentro de ella se distinguen:

- **Aseguramiento de calidad — QA/SQA:** establece y evalúa procesos, estándares y actividades para proporcionar confianza en que se cumplirán los requisitos de calidad.
- **Planificación de calidad:** determina qué calidad se busca para un proyecto o producto y cómo se evaluará.
- **Control de calidad — QC:** examina los resultados para detectar problemas y comprobar el cumplimiento de los criterios establecidos.

Por ejemplo, definir un procedimiento de revisión forma parte del aseguramiento; decidir qué componentes se revisarán y cuándo corresponde a la planificación; examinar esos componentes y registrar defectos corresponde al control. **Testing es una actividad de evaluación del producto dentro de un conjunto más amplio de actividades de calidad.**

2. IMPORTANCIA DE TRABAJAR PARA Y CON CALIDAD. VENTAJAS Y DESVENTAJAS

Trabajar **para la calidad** implica establecer objetivos y criterios para obtener un producto adecuado. Trabajar **con calidad** implica incorporar prácticas que permitan alcanzarlos durante todo el desarrollo.

La calidad debe construirse desde los requisitos y el diseño. Las pruebas ayudan a descubrir problemas, pero encontrar defectos al final obliga a corregir trabajo ya realizado y puede afectar varias partes del sistema. Por eso, la prevención y la detección temprana son fundamentales.

Ventajas

- **Menos retrabajo:** se evitan o descubren problemas antes de que se propaguen.
- **Menores costos de mantenimiento y soporte:** disminuyen las correcciones posteriores a la entrega.
- **Mayor satisfacción y confianza del cliente:** el producto responde mejor a sus necesidades.
- **Mayor previsibilidad:** los controles aportan información para gestionar riesgos y compromisos.
- **Mayor facilidad para evolucionar el producto:** un diseño y un código comprensibles facilitan los cambios.
- **Menor exposición a fallas:** se reducen riesgos para la operación y los usuarios.

Costos y dificultades

Trabajar con calidad requiere inversión en capacitación, planificación, revisiones, pruebas, herramientas y seguimiento. También puede generar resistencia al cambio o parecer más lento inicialmente.
Si las prácticas se aplican sin considerar el contexto, pueden producir documentación excesiva, controles redundantes y burocracia. Por eso, deben elegirse según los riesgos y necesidades del negocio. El objetivo es que el esfuerzo de calidad resulte útil y proporcional.

Costos de la calidad

Pressman distingue:

Tipo de costo	Ejemplo
Prevención	Capacitación, planificación de calidad y mejora de métodos.
Evaluación	Revisiones, auditorías, mediciones y pruebas.
Fallas internas	Corrección y repetición de trabajo antes de entregar el producto.
Fallas externas	Correcciones en producción, atención de reclamos y pérdida de confianza o clientes.

La decisión debe considerar tanto el costo de trabajar con calidad como **el costo de la falta de calidad**.

Principios para trabajar con calidad

La calidad es responsabilidad de todos y requiere personas capacitadas, apoyo de la dirección y objetivos claros. Sus actividades deben planificarse, integrarse al desarrollo y evaluarse mediante información útil.
Contar más pruebas no demuestra por sí solo una mejora de calidad: importa qué riesgos cubren, qué problemas detectan y cómo se utiliza esa información para corregir y prevenir defectos.

3. ACTIVIDADES RELACIONADAS CON EL ASEGURAMIENTO DE CALIDAD DE SOFTWARE

El aseguramiento de calidad acompaña el ciclo de vida completo. Pressman identifica actividades de planificación, supervisión, evaluación, registro y comunicación:
Preparar el plan de aseguramiento de calidad. Define estándares aplicables, evaluaciones, revisiones, auditorías, responsables y procedimientos para informar y seguir problemas.
Participar en la definición y adaptación del proceso. Se comprueba que la forma de trabajo elegida sea compatible con las políticas organizacionales y las necesidades del proyecto.
Evaluar las actividades de desarrollo. Se revisan requisitos, diseño, programación, integración y pruebas para comprobar que se realizan de acuerdo con los procedimientos establecidos.
Evaluar productos de trabajo. Se examinan documentos, modelos, código y otros resultados seleccionados para identificar incumplimientos.
Registrar y seguir desviaciones. Los problemas deben documentarse, asignarse a responsables y controlarse hasta su resolución.
Informar a la dirección. Se proporciona visibilidad sobre el estado de la calidad y se elevan los problemas que requieren decisiones.
Analizar métricas y causas. La información sobre defectos e incumplimientos permite identificar problemas recurrentes y mejorar el proceso.

Plan de aseguramiento de calidad de software — PACS

El PACS establece **qué actividades de aseguramiento se realizarán, sobre qué elementos, cuándo, por quién y con qué criterios**. Puede incluir:

- Objetivos y alcance.
- Estándares y procedimientos aplicables.
- Roles, responsabilidades y recursos.
- Revisiones y auditorías previstas.
- Registros e informes que se producirán.
- Procedimientos para tratar desviaciones y verificar su cierre.

El **plan de pruebas** desarrolla específicamente la evaluación mediante pruebas y puede relacionarse con el PACS, pero ambos tienen alcances diferentes.

Grupo de aseguramiento de calidad — GAC

El GAC proporciona una evaluación objetiva y ayuda a que el equipo alcance sus objetivos de calidad. Debe contar con suficiente independencia para informar problemas aunque exista presión por entregar.
La filmina recomienda que no dependa del gerente del proyecto que evalúa y que tenga acceso a una dirección interesada en la calidad. Esta independencia **complementa la responsabilidad del equipo de desarrollo**, que sigue siendo responsable de la calidad de su trabajo.

4. PRINCIPALES MODELOS DE CALIDAD: CMMI, SPICE E ISO

Los modelos y normas proporcionan referencias para **definir, evaluar y mejorar procesos o productos**. Es necesario distinguir su alcance: algunos se concentran en la capacidad del proceso, otros en la gestión de calidad y otros en las características del producto.

CMMI

CMMI — Capability Maturity Model Integration reúne buenas prácticas para mejorar y evaluar procesos organizacionales. Orienta sobre los objetivos que deben alcanzarse; la organización determina cómo implementarlos en su contexto.
Las filminas desarrollan principalmente las constelaciones de CMMI utilizadas en la versión 1.3:

- **CMMI-DEV:** desarrollo de productos y sistemas.
- **CMMI-ACQ:** adquisición de productos y servicios.
- **CMMI-SVC:** prestación de servicios.

Representación por etapas: madurez organizacional

Establece un camino de mejora mediante niveles de madurez:

Nivel	Características principales
1. Inicial	Procesos poco controlados; resultados dependientes del esfuerzo individual.
2. Administrado	Los proyectos planifican, ejecutan, supervisan y controlan sus procesos.

3. Definido	Existen procesos organizacionales establecidos que se adaptan a cada proyecto.
4. Administrado cuantitativamente	Se utilizan mediciones y análisis cuantitativo para gestionar el desempeño.
5. Optimizado	Se mejora continuamente mediante análisis de causas e introducción de mejoras.

La madurez describe la institucionalización de prácticas dentro del alcance organizacional evaluado.

Representación continua: capacidad de los procesos

Permite seleccionar áreas de proceso y mejorar su capacidad según las prioridades organizacionales. La **capacidad** se refiere a un proceso o área particular; la **madurez** se refiere al conjunto organizacional evaluado.

Hay una diferencia entre versiones que conviene registrar: la filmina muestra una escala continua de **0 a 5**, pero **CMMI v1.3 utiliza cuatro niveles de capacidad: 0 incompleto, 1 realizado, 2 administrado y 3 definido**.

Áreas de proceso del nivel 2

La filmina incluye:

- Administración de requerimientos.
- Planificación de proyectos.
- Monitoreo y control de proyectos.
- Administración de acuerdos con proveedores.
- Medición y análisis.
- Aseguramiento de calidad del proceso y del producto — **PPQA**.
- Administración de configuración.

Evaluación y relación con Agile

La evaluación contrasta las prácticas y evidencias de la organización con el modelo. En el marco de CMMI v1.3 se utilizaban métodos de evaluación como SCAMPI.

CMMI y Agile pueden coexistir: los objetivos de gestión y mejora pueden alcanzarse mediante prácticas ágiles adaptadas al contexto. La propia bibliografía oficial de CMMI-DEV v1.3 contempla su aplicación con métodos ágiles. CMU Software Engineering Institute

SPICE

SPICE es un enfoque de **evaluación y mejora de la capacidad de los procesos**, históricamente asociado con ISO/IEC 15504. Permite conocer la situación actual, identificar debilidades y establecer objetivos de mejora.

Distingue dos dimensiones:

- **Dimensión de procesos:** qué procesos se evalúan y qué resultados deben producir.
- **Dimensión de capacidad:** qué tan adecuadamente se ejecutan, gestionan y mejoran esos procesos.

Por eso, debe evitarse describir sus niveles simplemente como “madurez de toda la organización”: su evaluación se centra en la **capacidad de procesos específicos**.

En la escala clásica de ISO/IEC 15504 se distinguen seis niveles:

Nivel	Significado
0. Incompleto	El proceso no se realiza o no alcanza su propósito.
1. Realizado	Alcanza su propósito.
2. Gestionado	Su ejecución y sus productos se planifican y controlan.
3. Establecido	Se utiliza un proceso definido y desplegado.
4. Predecible	Opera dentro de límites conocidos mediante control cuantitativo.
5. Optimizado	Se mejora continuamente para responder a los objetivos del negocio.

ISO: gestión de calidad y calidad de producto

ISO 9000 e ISO 9001

La familia ISO 9000 proporciona referencias para la gestión de calidad. ISO 9000 trata fundamentos y vocabulario; **ISO 9001 establece requisitos para un sistema de gestión de calidad**.

Su propósito es organizar la gestión para proporcionar resultados consistentes, atender requisitos y mejorar. Una certificación del sistema de gestión **no demuestra que cada producto de software esté libre de defectos**.

ISO/IEC 25000 — SQuaRE

Es una familia orientada a los requisitos y la evaluación de la calidad de sistemas y software. La filmina utiliza el modelo de **ISO/IEC 25010:2011**, que distingue calidad de producto y calidad en uso. Esa edición contiene ocho características de calidad de producto; es importante mantener la versión porque ediciones posteriores modifican el modelo.

Característica	Qué considera
Adecuación funcional	Completitud, corrección y pertinencia de las funciones.
Eficiencia de desempeño	Tiempos de respuesta, recursos utilizados y capacidad.
Compatibilidad	Coexistencia e intercambio de información con otros sistemas.
Usabilidad	Facilidad de aprendizaje, operación y acceso para los usuarios.
Fiabilidad	Disponibilidad, tolerancia a fallos y recuperación.
Seguridad	Protección de información y control de acceso.
Mantenibilidad	Facilidad para analizar, modificar y probar el producto.
Portabilidad	Adaptación, instalación y traslado a otros entornos.

La **calidad en uso** considera los resultados que obtiene el usuario en un contexto concreto. La filmina también menciona **ISO/IEC 25012** como referencia para calidad de datos.

Otros modelos incluidos en la filmina

McCall agrupa factores de calidad en tres perspectivas:

- **Operación:** corrección, fiabilidad, eficiencia, integridad y usabilidad.
- **Revisión:** mantenibilidad, flexibilidad y facilidad de prueba.
- **Transición:** portabilidad, reutilización e interoperabilidad.

El esquema **Barbacci/SEI** presentado en clase destaca el equilibrio entre desempeño, confiabilidad y seguridad. Ayuda a comprender que mejorar un atributo puede afectar a otro y que deben evaluarse las necesidades del conjunto.

Modelo IDEAL para implementar mejoras

IDEAL organiza un programa de mejora en cinco fases:

1. **Iniciar:** reconocer la necesidad y obtener apoyo organizacional.
2. **Diagnosticar:** analizar la situación actual y las oportunidades de mejora.
3. **Establecer:** definir prioridades, objetivos y un plan.
4. **Actuar:** probar e implementar las mejoras.
5. **Aprender:** evaluar resultados y utilizar lo aprendido en el siguiente ciclo.

Mientras CMMI y SPICE ayudan a evaluar y orientar mejoras, IDEAL aporta una estructura para llevar adelante el programa de cambio.

5. AUDITORÍAS AL SOFTWARE

¿Qué es una auditoría?

Una auditoría es una **evaluación sistemática, objetiva e independiente** que utiliza evidencias para determinar el cumplimiento de criterios establecidos: estándares, procedimientos, especificaciones o acuerdos.
Puede examinar procesos y productos de trabajo. Permite identificar incumplimientos, reconocer buenas prácticas y dar visibilidad a la dirección sobre cómo se trabaja.

Auditoría de proyecto

Evalúa si el proyecto aplica el proceso y los controles previstos. Examina, por ejemplo:

- Planificación, responsabilidades y seguimiento.
- Gestión y aprobación de requerimientos.
- Consistencia entre requisitos, diseño, código y pruebas.
- Gestión de cambios y configuración.
- Cumplimiento de revisiones y actividades de calidad.

Debe realizarse según lo establecido en el PACS y respaldar sus conclusiones con evidencia. 14 Auditorías de Sw

Auditoría al grupo de calidad

Aplicando el mismo criterio de evaluación objetiva, también debe comprobarse si **el grupo de calidad cumple sus propias responsabilidades**. Se revisa si planifica y realiza evaluaciones, utiliza criterios adecuados, registra desviaciones, informa resultados y verifica acciones correctivas.
Esto evita asumir que las actividades de QA son eficaces solo porque existe un grupo encargado de realizarlas. Sus planes, registros e informes también deben poder evaluarse; estas son responsabilidades que Pressman asigna al grupo de aseguramiento.

Auditorías de configuración

Auditoría funcional: contrasta el producto construido y las evidencias de verificación con los requisitos especificados. Comprueba que las capacidades requeridas estén implementadas y que exista trazabilidad.
Auditoría física: comprueba que los elementos que integran la configuración entregada correspondan con su documentación y versiones identificadas. Incluye la consistencia entre código, ejecutables, documentación y otros elementos de la entrega.

Proceso de auditorías

Roles y responsabilidades

Rol	Responsabilidad principal
Responsable de SQA	Planificar auditorías, asignar recursos y asegurar el tratamiento de las no conformidades.
Auditor	Comunicar el alcance, reunir y analizar evidencias, informar resultados y verificar el seguimiento.
Auditado	Proporcionar evidencia, participar, responder al informe y realizar las acciones acordadas.

El auditor debe mantener objetividad y una comunicación respetuosa. El propósito es evaluar el cumplimiento y mejorar el trabajo. 14 Auditorías de Sw

Etapas

1. Preparación y planificación

Se definen objetivo, alcance, criterios, participantes, fechas y recursos. El auditor revisa información preliminar y prepara preguntas y listas de comprobación.

2. Ejecución

Comienza con una reunión de apertura para confirmar el alcance y la dinámica. Luego se realizan entrevistas, se examinan registros y productos y se contrastan evidencias con los criterios.

3. Análisis e informe

El auditor evalúa los hallazgos y comprueba que sus conclusiones tengan respaldo. En la reunión de cierre comunica los resultados y aclara dudas. Después emite el informe final.

4. Seguimiento

El auditado propone acciones con responsables y fechas. Se comprueba su realización y eficacia antes de cerrar las desviaciones. Informar que algo se corrigió debe acompañarse de evidencia de la corrección.

Técnicas y herramientas

Se utilizan entrevistas, observación, muestreo, revisión de registros, checklists y herramientas de seguimiento. Un checklist orienta la evaluación, pero las respuestas deben contrastarse con evidencia.
Los registros de cambios, resultados de pruebas, revisiones y versiones permiten reconstruir lo ocurrido y verificar el cumplimiento.

Resultados e informe

La filmina distingue:

- **Buenas prácticas:** resultados o formas de trabajo especialmente efectivos.
- **Desviaciones:** incumplimientos que requieren un plan de acción.

- **Observaciones:** condiciones que conviene mejorar o que podrían generar futuras desviaciones.

El informe incluye identificación, objetivo, alcance, participantes, criterios, evidencias, resultados y acciones solicitadas. También pueden medirse duración, esfuerzo, cantidad de desviaciones y tiempo de resolución.

6. CALIDAD DE PRODUCTO: PLANIFICACIÓN Y REALIZACIÓN DE PRUEBAS

¿Qué es testing?

Testing comprende actividades destinadas a **evaluar el software, descubrir problemas y aportar información sobre su calidad y sus riesgos**.
Myers enfatiza que debe buscarse activamente dónde el programa contradice lo esperado, incluyendo entradas inválidas y comportamientos no deseados. Las pruebas también permiten obtener confianza sobre los aspectos evaluados, pero **no demuestran la ausencia de todos los defectos**.
El testing busca descubrir defectos cuya presencia se supone posible en el software. Puede poner en evidencia los problemas encontrados y aportar información sobre la calidad, pero no garantiza que el producto esté libre de otros defectos. Por eso, superar las pruebas no demuestra por sí solo la calidad completa del producto.

Verificación y validación

Verificación: controla que el producto de trabajo cumple las especificaciones y condiciones establecidas. Pregunta: **¿estamos construyendo el producto correctamente?**
Validación: controla que el producto satisface las necesidades de sus usuarios. Pregunta: **¿estamos construyendo el producto correcto?**
Por ejemplo, verificar un cálculo comprueba que aplica la fórmula especificada; validarlo comprueba que esa fórmula y sus resultados sirven para la necesidad del negocio. Ambas actividades pueden realizarse durante todo el ciclo de vida mediante revisiones y pruebas.

Error, defecto y falla

Una distinción habitual es:

- **Error:** equivocación humana. El problema se detecta dentro de la misma etapa en la que se originó
- **Defecto:** problema introducido en un requisito, diseño, código u otro producto de trabajo. Se propaga y se encuentra en una etapa posterior
- **Falla:** comportamiento incorrecto que se manifiesta durante la ejecución. Un defecto puede permanecer sin manifestarse hasta que se presentan determinadas condiciones.

Por ejemplo, interpretar mal una regla puede introducir una condición incorrecta en el código y provocar una falla cuando se utiliza cierto dato.
La terminología varía entre fuentes. Pressman utiliza en algunos apartados una distinción basada en si el problema se detecta antes o después de la entrega; la filmina de revisiones presenta otra convención para el origen del problema. Conviene registrar esa diferencia al estudiar.

Planificación de pruebas

La planificación define qué evaluar, por qué, cómo, cuándo y con qué recursos. Debe partir de los objetivos del producto y sus riesgos.
Un **plan de pruebas** puede incluir:

- Alcance y elementos que se probarán.
- Riesgos y prioridades.
- **Niveles, tipos y técnicas de prueba.**
- Ambientes, datos y herramientas.
- Responsables, recursos y calendario.
- Criterios para comenzar, suspender, reanudar y finalizar.
- Tratamiento y seguimiento de defectos.
- Informes y evidencias que se producirán.

Las actividades se priorizan por riesgo: las funciones con mayor impacto o probabilidad de falla requieren más atención.

Proceso de pruebas

Planificación y control: establecer objetivos y estrategia, supervisar resultados y ajustar el trabajo.
Análisis y diseño: revisar requisitos y otros documentos, identificar condiciones de prueba y diseñar casos, datos y ambientes.
Preparación y ejecución: implementar las pruebas, preparar el entorno, ejecutar los casos y comparar resultados reales con esperados.
Evaluación y cierre: analizar defectos y cobertura, comprobar criterios de finalización, informar riesgos y conservar resultados y lecciones aprendidas.

Ciclo de pruebas

Un ciclo de pruebas comprende la ejecución del conjunto de casos seleccionado sobre una **misma versión del software**, utilizando un entorno y datos identificados. Se registran resultados, defectos y condiciones de ejecución. Cuando las correcciones generan una nueva versión, se comprueba que los defectos fueron solucionados mediante pruebas de **confirmación** y se realizan pruebas de **regresión** para detectar efectos sobre comportamientos existentes. El **proceso de pruebas** comprende planificación, diseño, ejecución y evaluación; un **ciclo** es una ejecución concreta del conjunto definido.

Casos de prueba y reporte de defectos

Un caso de prueba describe **qué se quiere comprobar, bajo qué condiciones, con qué datos y qué resultado se espera**. Puede incluir identificación, objetivo, precondiciones, pasos y resultado esperado.
Los casos se derivan de requisitos, historias de usuario, criterios de aceptación, reglas de negocio y otros documentos que expresan el comportamiento esperado. Deben permitir identificar **qué se comprueba y contra qué criterio se compara el resultado**. Un caso debe tener un objetivo claro. Puede comprobar varias condiciones relacionadas, pero conviene separar objetivos diferentes cuando su combinación dificulta interpretar el resultado o localizar el problema.
Diseñar una prueba consiste en determinar qué situación se evaluará, con qué datos y qué resultado se espera. Ejecutarla consiste en aplicar el caso a una versión del software y registrar el resultado obtenido. Por eso, pueden diseñarse casos antes de tener el sistema construido.
Ejemplo: si una reserva admite entre 1 y 5 personas, un caso introduce 6 y espera que se rechace la operación con un mensaje adecuado. Al encontrar un problema, el reporte debe permitir comprenderlo e investigarlo: versión, entorno, precondiciones, pasos, datos, resultado esperado, resultado obtenido y evidencia.

Severidad y prioridad de los defectos

Severidad	Significado
Bloqueante	Impide utilizar el sistema o continuar la actividad evaluada.
Crítico	Impide realizar una funcionalidad importante y no existe una alternativa.
Mayor	Impide alcanzar el objetivo por el recorrido previsto, pero existe una alternativa.

Menor	Afecta parcialmente el comportamiento sin impedir la operación principal.
Cosmético	Afecta presentación, ubicación, tipografía u otros aspectos de forma.

La **prioridad** indica la urgencia con que debe resolverse: urgente, alta, media o baja. Se determina considerando las necesidades del negocio, los compromisos y el contexto.

Severidad y prioridad no son equivalentes. Un problema de presentación puede tener alta prioridad si afecta una demostración importante; una falla grave en una función poco utilizada puede tener menor urgencia inmediata. Estas categorías corresponden al esquema explicado en clase; no deben confundirse con la clasificación de hallazgos de las inspecciones.

Niveles de prueba

Los niveles distinguen **qué alcance del software se evalúa**:

Nivel	Qué comprueba	Ejemplo
Unitario o componente	Una unidad pequeña de comportamiento.	Una función que calcula el importe.
Integración	Interacciones entre componentes o sistemas.	El servicio guarda correctamente una reserva en la base de datos.
Sistema	El comportamiento del sistema completo.	Registrar, consultar y cancelar una reserva.
Aceptación	Adecuación a necesidades y criterios del negocio o usuario. A partir de ahi se habilita poner en produccion el incremento.	Los responsables comprueban que el circuito les permite operar como necesitan.

Las **pruebas unitarias** son realizadas principalmente por los desarrolladores durante la implementación. Las **pruebas de integración** comprueban interfaces e interacciones y pueden ejecutarse automáticamente dentro de la integración continua.

Las **pruebas de sistema** evalúan una versión integrada del producto. Conviene que incorporen una perspectiva diferente de la de sus autores para descubrir problemas que estos podrían pasar por alto.

Las **pruebas de aceptación de usuario — UAT** permiten que usuarios o representantes del negocio comprueben si la solución satisface sus necesidades, con acompañamiento del equipo.

Cada nivel busca problemas diferentes. Superar las pruebas unitarias no reemplaza la evaluación de las interacciones ni del producto completo.

Ambientes para construir y probar software

Separar ambientes permite realizar cambios y pruebas sin afectar la operación real. En clase se presentó una organización mínima de tres ambientes y una separación adicional de preproducción:

- **Desarrollo:** utilizado para construir y comprobar cambios durante la implementación.
- **Testing:** utilizado para evaluar versiones identificadas con datos y condiciones controladas.
- **Preproducción o aceptación:** semejante a producción; permite realizar aceptación y verificaciones previas a la entrega.
- **Producción:** ambiente donde los usuarios operan con el sistema real.

La distribución en tres o cuatro ambientes es una referencia presentada en clase, no una cantidad universal.Para interpretar una prueba deben conocerse la versión, configuración, datos y recursos utilizados. Esto es especialmente importante en pruebas de rendimiento, carga y compatibilidad, cuyos resultados dependen del entorno.

Tipos de prueba

Los tipos de prueba se seleccionan según el aspecto que se necesita evaluar. Siguiendo la organización utilizada en clase, se agrupan según atiendan requerimientos funcionales o no funcionales.

Pruebas orientadas a requerimientos funcionales

- **Operación normal:** comprueban el uso previsto del sistema y sus resultados esperados; incluyen el camino feliz.
- **Pruebas negativas:** comprueban que el sistema rechace datos, acciones o usos no permitidos. Son satisfactorias cuando impiden correctamente la operación indebida.
- **Proceso de negocio:** evalúan un circuito completo que conecta varias funcionalidades. Por ejemplo, seleccionar una actividad, inscribirse y recibir la confirmación.
- **Documentación:** comprueban que manuales e instrucciones sean correctos y consistentes con el funcionamiento del producto.

Pruebas orientadas a requerimientos no funcionales

- **Desempeño o performance:** evalúan tiempos de respuesta y utilización de recursos bajo condiciones determinadas.
- **Escala completa o carga:** comprueban el funcionamiento con la demanda y configuración comprometidas, como determinada cantidad de usuarios concurrentes.
- **Estrés:** someten al sistema a condiciones extremas o superiores a las previstas para observar sus límites, degradación y recuperación.
- **Usabilidad:** evalúan cuánto esfuerzo requiere alcanzar un objetivo, considerando comprensión, tiempo, pasos y experiencia de uso.
- **Accesibilidad:** comprueban que personas con distintas capacidades puedan percibir, comprender y operar el sistema.
- **Mantenimiento:** evalúan la facilidad para analizar, modificar y comprobar cambios.
- **Fiabilidad:** evalúan la consistencia del funcionamiento y la confianza que puede depositarse en sus resultados bajo las condiciones previstas.
- **Portabilidad:** comprueban la adaptación a los entornos requeridos, como sistemas operativos, navegadores o dispositivos.
- **Seguridad:** comprueban protección de información, autenticación y permisos. Cuando corresponda al sistema completo, también se consideran controles físicos.
- **Interfaz de usuario:** evalúan controles, mensajes, combinaciones, habilitación de opciones y correspondencia con las reglas de negocio. Pueden comprender aspectos funcionales y de usabilidad.

Confirmación o retest: comprueba que un defecto específico fue corregido.

Regresión: comprueba que un cambio no perjudicó comportamientos existentes. Su alcance depende del impacto y los riesgos del cambio.

Smoke test: conjunto breve de comprobaciones esenciales para decidir si una versión está suficientemente estable para continuar probándola.

Una prueba puede clasificarse simultáneamente por nivel y tipo: por ejemplo, una **prueba de sistema de rendimiento**.

Técnicas para diseñar pruebas

Caja negra

Diseña casos a partir del comportamiento esperado y las especificaciones, sin depender de la estructura interna. No tenemos acceso al proceso que transforma entradas en salida. Es dinámico.

Partición de equivalencias

Consiste en dividir el dominio de entradas en clases cuyos valores se espera que tengan un comportamiento semejante. Se identifican clases válidas e inválidas y se seleccionan datos representativos, evitando probar todos los valores posibles.

El procedimiento tiene dos pasos:

1. **Identificar las clases:** analizar cada condición del requisito, establecer sus clases válidas e inválidas y asignarles identificadores.
2. **Diseñar los casos:** elegir valores que cubran las clases identificadas y definir el resultado esperado.

Un caso puede combinar varias clases válidas compatibles. Para probar una clase inválida, conviene mantener las demás entradas válidas: así se evita que un error oculte otro y se facilita interpretar el resultado.

Análisis de valores límite

Complementa la partición de equivalencias seleccionando valores en las fronteras y sus vecinos, donde suelen aparecer errores de comparación o de cambio de tramo.

Para un intervalo de números enteros válido entre **a y b inclusive**, se consideran:

a − 1, a, a + 1, b − 1, b, b + 1.

La separación depende del dato: una unidad para enteros, un centavo para importes o un día para determinadas fechas. Debe respetarse si los extremos son inclusivos o exclusivos.

- **Tablas de decisión:** organizan combinaciones de condiciones y las acciones esperadas.
- **Transiciones de estado:** comprueban cambios de estado permitidos y rechazados.

Para una cantidad válida de 1 a 5, las clases serían menores que 1, entre 1 y 5 y mayores que 5. Los valores límite pueden incluir **0, 1, 2, 4, 5 y 6**.

Caja blanca

Diseña casos a partir de la estructura interna del código. Es estatico. Puede analizar:

- **Cobertura de sentencias:** qué instrucciones se ejecutaron.
- **Cobertura de decisiones o ramas:** qué resultados de las decisiones se recorrieron.
- **Cobertura de condiciones:** qué valores tomaron las condiciones individuales.
- **Caminos y bucles:** qué recorridos y repeticiones fueron evaluados.

Una cobertura del 100 % de sentencias no demuestra que se probaron todas las combinaciones ni que el comportamiento sea correcto. La **complejidad ciclomática** indica la cantidad de caminos linealmente independientes del flujo de control.

La **caja negra** utiliza especificaciones y comportamientos observables para diseñar pruebas. La **caja blanca** utiliza conocimiento de la estructura interna y la lógica del código.

La evaluación **estática** examina productos de trabajo sin ejecutar el software, mediante revisiones o análisis. La evaluación **dinámica** ejecuta el software y compara sus resultados con lo esperado.

Basadas en la experiencia

La **adivinanza de defectos** utiliza conocimientos sobre errores frecuentes para orientar casos.

El **testing exploratorio** combina aprendizaje, diseño y ejecución: el tester investiga el producto y utiliza los resultados para decidir qué comprobar después. Puede organizarse mediante objetivos y sesiones con registro de hallazgos.

Herramientas de testing

Las herramientas apoyan distintas actividades:

Categoría	Utilidad
Gestión de pruebas	Organizar casos, ejecuciones y resultados.
Seguimiento de defectos	Registrar, priorizar y controlar problemas.
Frameworks de pruebas automatizadas	Ejecutar comprobaciones y comparar resultados.
Medición de cobertura	Identificar partes del código ejecutadas por las pruebas.
Pruebas de API o interfaz	Comprobar servicios o interacciones de usuario.
Carga y rendimiento	Generar demanda y medir respuestas.
Integración continua	Ejecutar verificaciones con los cambios de código.
Datos y dobles de prueba	Preparar condiciones y controlar dependencias.

La automatización debe aportar resultados confiables y mantenerse junto con el producto. También requiere inversión en datos, ambientes y diagnóstico de fallas.

Principios del testing y finalización

1. Las pruebas muestran la presencia de defectos, pero no demuestran su ausencia.
2. Las pruebas exhaustivas son inviables para sistemas no triviales.
3. Probar tempranamente reduce problemas y retrabajo.
4. Los defectos suelen concentrarse en algunas partes del sistema.
5. Repetir siempre las mismas pruebas puede dejar de revelar nuevos problemas.
6. La estrategia depende del contexto.
7. Un producto sin defectos detectados puede seguir siendo inútil si no satisface necesidades. 09 Testing de Software

Se decide finalizar según criterios acordados: cobertura relevante, resultados obtenidos, defectos pendientes y riesgos aceptados. **La falta de tiempo puede detener las pruebas, pero no demuestra que se haya alcanzado la calidad buscada.**

7. TÉCNICAS Y HERRAMIENTAS PARA REVISIONES TÉCNICAS

¿Qué es una revisión técnica?

Una revisión técnica examina un producto de trabajo para detectar defectos, inconsistencias e incumplimientos. Puede aplicarse a requisitos, diseños, código, planes y casos de prueba.

Es una actividad **estática**: no requiere ejecutar el programa. Por eso permite encontrar problemas antes de disponer de software funcionando y complementa las pruebas dinámicas.

Tipos de revisión

Revisión informal o entre pares: uno o más colegas examinan el trabajo y realizan comentarios. Tiene menor formalidad, pero debe dejar claros los problemas y su tratamiento.

Walkthrough o recorrida: el autor guía a los participantes por el producto. Se formulan preguntas, se examinan ejemplos y se detectan problemas. También facilita la comprensión y transferencia de conocimiento.

Inspección: utiliza un proceso formal con roles, preparación individual, criterios, registros y seguimiento. Busca detectar defectos de manera sistemática.

Roles de una inspección

- **Autor:** crea o mantiene el producto y realiza las correcciones.
- **Moderador:** planifica, coordina y verifica el seguimiento.

- **Lector:** presenta o recorre el material.
- **Anotador:** registra hallazgos.
- **Inspectores:** estudian el producto y buscan problemas.

La distribución concreta puede variar según el tamaño del equipo y el proceso utilizado.

Proceso de inspección

1. **Planificación:** seleccionar material, participantes y criterios.
2. **Presentación general:** explicar el contexto cuando sea necesario.
3. **Preparación individual:** estudiar el material antes de la reunión.
4. **Reunión:** reunir, discutir y registrar los defectos.
5. **Corrección:** el autor trata los problemas.
6. **Seguimiento:** comprobar las correcciones y decidir si hace falta reinspeccionar.

Durante la revisión se debe **evaluar el producto y mantener el foco en los problemas**, evitando convertir la reunión en una evaluación personal o en una discusión extensa sobre todas las soluciones posibles.

Clasificación de hallazgos

Los problemas encontrados durante una inspección pueden clasificarse en:

- **Mayores:** pueden provocar fallas operacionales o resultados incorrectos. Por ejemplo, una regla de negocio mal especificada.
- **Menores:** afectan la claridad, el mantenimiento o el cumplimiento de estándares, sin provocar directamente una falla operacional. Por ejemplo, nombres de variables que incumplen las convenciones acordadas.
- **Cosméticos:** problemas de presentación que no alteran el significado ni el funcionamiento, como errores tipográficos sin consecuencias.

La clasificación depende del impacto del problema: un error de escritura que modifica el sentido de un requisito puede ser mayor.

Resultados de la inspección

La inspección puede concluir en **aceptar el producto sin modificaciones**, **aceptarlo provisionalmente sujeto a correcciones** o **rechazarlo y solicitar correcciones y una nueva inspección**. Los hallazgos y las decisiones se registran en un informe, y el seguimiento verifica que las correcciones se hayan realizado.

Ventajas, dificultades y herramientas

Las revisiones detectan defectos tempranamente, pueden aplicarse a productos incompletos y favorecen el conocimiento compartido. Requieren preparación, capacitación y tiempo; sin una buena organización pueden convertirse en reuniones poco efectivas.
Se apoyan en checklists, guías de lectura, estándares, plantillas y registros de hallazgos. Las herramientas de revisión y análisis estático pueden ayudar a encontrar problemas, pero sus resultados necesitan interpretación.
Pueden medirse densidad de defectos, esfuerzo de revisión, esfuerzo por defecto, correcciones y re inspecciones. Las métricas deben utilizarse para mejorar la revisión y el proceso.

8. TESTING EN AMBIENTES ÁGILES

Enfoque general

En ambientes ágiles, el testing se integra al desarrollo desde el comienzo. Las pruebas ayudan a **comprender necesidades, orientar la construcción y obtener retroalimentación rápida**. La calidad es responsabilidad de todo el equipo. Programadores, testers y representantes del negocio colaboran para definir ejemplos, construir, evaluar y mejorar el producto durante cada iteración.
El testing ágil conserva la necesidad de descubrir problemas y evaluar el producto. Cambia principalmente **cómo se organizan las actividades, cuándo se realizan y cómo colaboran las personas**.Las pruebas acompañan el desarrollo para prevenir problemas y reducir el tiempo de retroalimentación. El tester necesita comprender el negocio y contribuir a validar que el producto genere valor. Encontrar defectos es una contribución a la construcción de un mejor sistema. La automatización permite repetir comprobaciones y regresión con frecuencia dentro del tiempo disponible. Su beneficio depende de la relevancia y calidad de las pruebas: aumentar su cantidad no garantiza automáticamente mayor calidad.

Manifiesto del testing ágil

El manifiesto del testing ágil propone valorar:

- **Probar durante el desarrollo** sobre concentrar las pruebas al final.
- **Prevenir defectos** sobre limitarse a encontrarlos posteriormente.
- **Comprender lo que se prueba** sobre comprobar funcionalidades mecánicamente.
- **Construir un mejor producto** sobre considerar que encontrar fallas es el único objetivo.
- **La responsabilidad de todo el equipo** sobre delegar el testing exclusivamente al tester.

Estas prioridades orientan las pruebas hacia la colaboración, la prevención y la retroalimentación temprana. No eliminan las pruebas finales ni la búsqueda de defectos; las integran al trabajo continuo del equipo.

Principios destacados por la filmina

- Comenzar a probar tempranamente.
- Integrar el testing al trabajo continuo.
- Compartir la responsabilidad de las pruebas.
- Reducir el tiempo entre un cambio y su evaluación.
- Expresar expectativas mediante pruebas.
- Mantener el código limpio y corregir defectos rápidamente.
- Producir documentación útil y proporcional.
- Incluir las pruebas en la definición de terminado.
- Utilizar pruebas para orientar el desarrollo.

Rol del tester

El tester aporta conocimientos para aclarar requisitos, identificar riesgos, diseñar pruebas y explorar el comportamiento del producto. También ayuda a expresar necesidades del negocio mediante ejemplos verificables.
Trabaja junto con los programadores y los interesados. La responsabilidad compartida permite utilizar especialidades sin crear una separación rígida entre quienes construyen y quienes evalúan.

Prácticas de testing ágil

La filmina destaca:

- Pruebas unitarias y de integración automatizadas.

- Pruebas de regresión de sistema automatizadas.
- Testing exploratorio.
- TDD.
- ATDD.
- Control de versiones de las pruebas junto con el código.

TDD — Desarrollo conducido por pruebas

Es una práctica de desarrollo y diseño basada en ciclos pequeños:

1. **Rojo:** escribir una prueba que falle porque falta el comportamiento.
2. **Verde:** implementar lo necesario para hacerla pasar.
3. **Refactorizar:** mejorar la estructura manteniendo las pruebas exitosas.

Las pruebas proporcionan retroalimentación y una protección para cambiar el código. TDD no sustituye los otros niveles y tipos de evaluación del producto.

ATDD — Desarrollo conducido por pruebas de aceptación

Antes de implementar, representantes del negocio, testers y programadores acuerdan ejemplos que expresan el comportamiento esperado. Estos ejemplos se convierten en pruebas que orientan el desarrollo y permiten comprobar condiciones de satisfacción.
TDD se concentra principalmente en el diseño y comportamiento de unidades pequeñas; ATDD ayuda a construir una comprensión compartida de las expectativas del negocio.

Cuadrantes de Agile Testing

Los cuadrantes de Crispin y Gregory combinan dos perspectivas: pruebas orientadas a tecnología o negocio, y pruebas que apoyan la construcción o evalúan críticamente el producto.
Los cuadrantes no son fases ni un orden de ejecución. Ayudan a reconocer qué evaluaciones se necesitan según los riesgos y objetivos del producto.

Cuadrante	Propósito	Ejemplos
Q1: tecnología, apoyo al equipo	Orientar y comprobar la construcción técnica.	Pruebas unitarias y de componentes.
Q2: negocio, apoyo al equipo	Aclarar y comprobar el comportamiento esperado.	Ejemplos, pruebas de historias, funcionales y prototipos.
Q3: negocio, evaluación del producto	Investigar si el producto satisface realmente al usuario.	Exploratorias, usabilidad y aceptación de usuario.
Q4: tecnología, evaluación del producto	Evaluar atributos y riesgos técnicos.	Rendimiento, carga, seguridad y otros atributos de calidad.

Pirámide de automatización

La pirámide orienta la distribución de las pruebas automatizadas:

- **Base:** muchas pruebas unitarias y de componentes, rápidas y focalizadas.
- **Nivel intermedio:** pruebas de servicios, API y aceptación por debajo de la interfaz.
- **Parte superior:** una cantidad menor de pruebas mediante la interfaz gráfica, generalmente más lentas y costosas de mantener.

Se busca evaluar cada comportamiento en el nivel que proporcione información útil con un costo razonable. Las pruebas manuales, exploratorias y de aceptación humana complementan esta estrategia.

Automatización, integración continua y Definition of Done

La automatización facilita ejecutar regresión con frecuencia, obtener resultados repetibles y detectar problemas poco después de introducir cambios. La integración continua combina integración frecuente, construcción y pruebas automáticas.
La **Definition of Done** establece las condiciones para considerar terminado el trabajo. Puede exigir pruebas exitosas, revisión, integración y cumplimiento de criterios de calidad. Las actividades necesarias para alcanzar esa definición deben realizarse dentro del trabajo del equipo y de la iteración.

Desafíos culturales

La transición requiere dejar de tratar los defectos como un conflicto entre personas y utilizarlos como información para mejorar. También requiere aprender a colaborar, compartir conocimientos y comunicar resultados y riesgos.
La independencia de criterio sigue siendo valiosa, pero se combina con colaboración y responsabilidad colectiva. Las revisiones, auditorías y modelos de calidad pueden adaptarse a esta forma de trabajo.

Diferencias que conviene recordar

Conceptos	Diferencia principal
QA / Testing	QA abarca el aseguramiento de calidad; testing evalúa el producto mediante actividades de prueba.
Proceso / Producto	Uno describe cómo se trabaja; el otro, qué calidad tienen los resultados.
Verificación / Validación	Cumplimiento de especificaciones / satisfacción de necesidades.
Auditoría / Revisión técnica	Evaluación objetiva de cumplimiento / examen técnico para detectar problemas.
Revisión / Prueba dinámica	Examinar sin ejecutar / evaluar mediante ejecución.
Nivel / Tipo / Técnica	Alcance evaluado / propósito evaluado / forma de diseñar las pruebas.
Confirmación / Regresión	Comprobar una corrección / comprobar que el cambio no dañó otros comportamientos.
Cuadrantes / Pirámide	Perspectivas y propósitos de evaluación / distribución de la automatización.
Capacidad / Madurez	Desempeño de procesos específicos / institucionalización organizacional dentro del alcance evaluado.

9. TDD — LEYES Y PATRONES DEL DESARROLLO GUIADO POR PRUEBAS

TDD (*Test-Driven Development*) es una práctica de desarrollo que combina escribir las pruebas primero y refactorizar el código. Se trabaja en pasos pequeños: antes de implementar un comportamiento, se escribe una prueba automatizada —generalmente unitaria— que expresa lo esperado. Las pruebas orientan el diseño y proporcionan retroalimentación inmediata.

En la secuencia tradicional se suelen diseñar e implementar componentes antes de escribir sus pruebas unitarias; en TDD las pruebas preceden al código. Esto permite anticipar decisiones de diseño y encontrar problemas tempranamente.

Ciclo Red–Green–Refactor

El corazón de TDD consiste en repetir tres pasos:

1. **Red — Rojo:** escribir una prueba para un comportamiento pequeño y comprobar que falla porque todavía no está implementado.
2. **Green — Verde:** escribir el código mínimo necesario para que la prueba pase y comprobar que las anteriores siguen pasando.
3. **Refactor:** mejorar la estructura del código, eliminando duplicación y complejidad innecesaria, sin cambiar su comportamiento observable.

Después se agrega una nueva prueba y se repite el ciclo. Así, la funcionalidad y el diseño evolucionan gradualmente.

Secuencia práctica: escribir una prueba, hacerla compilar, comprobar que falla (Red), realizar un cambio mínimo, ejecutar las pruebas hasta obtener Green y refactorizar quitando duplicaciones. Luego se repite con el siguiente comportamiento.

Las tres leyes de TDD

- No escribir código de producción sin una prueba que falle.
- No escribir más código de prueba del necesario para producir ese fallo.
- No escribir más código de producción del necesario para hacer pasar la prueba.

Patrones principales

Los patrones ayudan a decidir **qué prueba escribir y cómo avanzar hacia una solución**.

Para comenzar o ampliar las pruebas, se utilizan casos pequeños, pruebas iniciales que exploran el comportamiento esperado y nuevos casos que cubren variantes. Una **prueba de regresión** reproduce un defecto descubierto para detectar si reaparece.

Patrones para escribir la prueba que falla (Red Bar)

- **One Step Test:** probar una única porción mínima de funcionalidad por vez.
- **Starter Test:** crear una prueba inicial para explorar cómo debería usarse la API o el componente.
- **Explanation Test:** formular comentarios o preguntas que aclaren qué comportamiento se espera.
- **Learning Test:** escribir pruebas para comprender el comportamiento de una biblioteca o API externa.
- **Another Test:** agregar otro caso que cubra una variante o condición todavía no verificada.
- **Regression Test:** reproducir mediante una prueba un defecto encontrado, para detectar si reaparece.

Patrones para hacer pasar la prueba (Green Bar)

Para alcanzar el estado verde, la filmina destaca:

- **Fake It:** utilizar temporalmente una respuesta fija o una solución muy simple para hacer pasar la prueba.
- **Triangulación:** agregar casos con datos diferentes que obliguen a generalizar la solución.
- **Implementación obvia:** escribir directamente la solución cuando es clara y sencilla.
- **One to Many:** partir de una solución para un solo elemento y extenderla progresivamente para tratar colecciones.

Por ejemplo, ante una primera prueba de suma, devolver 5 puede hacer pasar el caso 2 + 3. Una segunda prueba, como 4 + 3 = 7, obliga a reemplazar esa respuesta fija por una solución general.

También se recomienda mantener pruebas independientes, una lista de casos pendientes y datos y resultados esperados fáciles de comprender.

Patrones de organización de pruebas

- **Test:** utilizar nombres de prueba que identifiquen con claridad el comportamiento evaluado.
- **Isolated Test:** mantener las pruebas independientes entre sí; ninguna debe depender del resultado de otra.
- **Test List:** escribir antes una lista de casos o comportamientos pendientes.
- **Test First:** expresar el comportamiento esperado en una prueba antes de escribir el código de producción.
- **Assert First:** definir primero la aserción o resultado esperado y después preparar los datos y pasos.
- **Test Data:** utilizar datos que faciliten comprender y seguir el caso de prueba.
- **Evident Data:** hacer evidente la relación entre las entradas y los resultados esperados.

La filmina también menciona familias complementarias: patrones de testing, patrones de diseño y patrones xUnit. No deben confundirse con las tres leyes del ciclo TDD.

Fuente de los patrones detallados: filminas de TDD de la cátedra, diapositivas 8 a 11.

Refactorización

Refactorizar consiste en **mejorar la estructura interna del software sin modificar su comportamiento observable**. Permite eliminar duplicación, simplificar el código, mejorar nombres y facilitar futuros cambios.

Se parte de pruebas exitosas, se realizan modificaciones pequeñas y se vuelven a ejecutar las pruebas. Esto ayuda a incorporar lo aprendido durante el desarrollo y evitar el deterioro del diseño.

Beneficios y alcance

TDD favorece código comprensible, componentes con **alta cohesión y bajo acoplamiento**, detección temprana de problemas y mayor confianza para modificar y mantener el software. Escribir primero las pruebas también obliga a pensar cómo se utilizará el componente.

Sin embargo, las pruebas exitosas **no garantizan por sí solas la calidad completa del producto**. TDD debe complementarse con pruebas de integración, aceptación, seguridad, rendimiento y otras evaluaciones necesarias según el contexto.